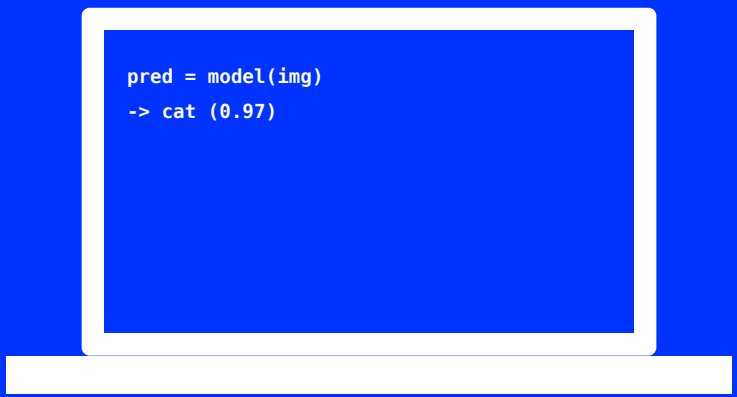
{ PISCINE IA }

# PROJECT 01

< CAT OR DOG? />

Special piscine - Build your first image classifier




v 1.0

# PROJECT 01

## The mission

Between two piscines, a **special project**: you build, from scratch, an AI model that looks at a photo and answers **cat** or **dog**, with a confidence level. Not an API call, not a ready-made notebook: you load the data, design the network, write the training loop, measure honestly, and ship a small program anybody can use.

You do the job **twice**, as in the main piscine: first a small convolutional network **built from zero** (random weights, your architecture), then a **pretrained** network adapted to the task. Comparing both is the real lesson.

| Day | Theme | You deliver |
|---|---|---|
| 1 | Understand the problem and the data | Exploration notebook, written answers |
| 2 | Data pipeline | Clean train / val / test split, transforms, DataLoaders |
| 3 | Your first CNN | `SmallCNN` that passes the sanity checks |
| 4 | Training | Hand-written training loop, curves, experiment log |
| 5 | Honest evaluation | Test results, confusion matrix, error analysis |
| 6 | Improve | Regularisation + pretrained ResNet18, comparison table |
| 7 | Ship it | `predict.py`, real-world test, model card |

> **i Tip:** Plan 3 hours a day, like the main piscine. Colab with a free GPU is enough for everything here. One run of the small CNN takes a few minutes, not hours.

## Core ideas in one page

Read this page now, and again after Day 4. You will be asked to explain each line without notes.

- ✓ **An image is numbers.** A colour photo of 128 x 128 pixels is a tensor of shape `3 x 128 x 128` (3 colour channels). The model never sees a cat, only numbers.
- ✓ **A model is a function with weights.** It maps the numbers of the image to two scores, one per class, called **logits**. At the start the weights are random, so the answers are random.
- ✓ **Softmax** turns the two logits into probabilities that sum to 1.
- ✓ **The loss** measures how wrong the model is. For classification we use cross-entropy: it is small when the model gives a high probability to the correct class.
- ✓ **Gradient descent.** Backpropagation computes how each weight should change to reduce the loss; the optimiser nudges all weights a little, many times.
- ✓ **Epoch / batch.** One epoch is one pass over the training set; a batch is the small group of images used for one weight update.
- ✓ **Convolution.** A small filter slides over the image and detects local patterns (edges, then textures, then ears or eyes in deeper layers). It reuses the same weights everywhere, which is why a CNN needs far fewer weights than a layer connected to every pixel.

- ✓ **Overfitting.** The model memorises the training images instead of learning cat vs dog. The sign: training accuracy keeps rising, validation accuracy stops or falls.
- ✓ **Three datasets.** Train: the model learns from it. Validation: you tune choices with it. Test: you look at it **once**, at the end, to get an honest number.

⚠ **Warning:** A model that says 97% on your photos has learned *patterns that separated cats from dogs in this dataset*. It has no idea what a cat is. This is why Day 7 tests it on photos it was never built for.

## Rules of this project

1. The 7 rules of the piscine apply: never copy code you cannot explain, one exercise per concept, explain without notes, document on GitHub, read the official docs, accept errors.
2. **The test set is opened once**, on Day 5 for your first model and on Day 6 for the final comparison. Every earlier decision is made on the validation set.
3. Fix the random seed (`42`) everywhere and say so in your README.
4. Log **every** training run in `experiments.csv`, including the bad ones.
5. Write each design answer and each `TODO` yourself first. After 20 minutes of being truly stuck you may ask an assistant for a hint, never for the full solution, and you note what it told you.

**i Tip:** Code skeletons in this document are valid Python. The `...` marks what **you** write. Anything you can run immediately is already complete.

## Task 00 - Repository and Colab

Create the repository **special01-cat-or-dog** (private is fine) with this layout, and a `.gitignore` that ignores `data/`, `*.pt`, `.venv/` and `__pycache__/`.

layout

```
special01-cat-or-dog/
  README.md
  requirements.txt
  experiments.csv          # one line per training run
  notebooks/
    01_explore.ipynb
    02_train_cnn.ipynb
    03_transfer.ipynb
  src/
    data.py                # transforms, split, loaders
    model.py               # SmallCNN
    train.py               # training loop, evaluation
    predict.py             # command line tool
  models/                  # saved weights (not in Git)
  MODEL_CARD.md
```

In Colab choose **Runtime > Change runtime type > T4 GPU**, then check that it is used:

colab cell

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())            # must print True
device = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.cuda.get_device_name(0) if device == "cuda" else "CPU only")
```

⚠ **Warning:** Colab files disappear when the session ends. Keep your code in `src/` and push it to GitHub; save weights to Google Drive or download them.

# Day 1 - Understand the problem and the data

Before any code that learns, you define precisely what you are asking the machine to do, and you look at your data with your own eyes. Most failed AI projects fail here.

## Task 01 - Think before you code

In `notebooks/01_explore.ipynb` (a text cell) or in `DESIGN.md`, answer in 3 to 5 sentences each. **No assistant for the first draft.**

1. What is the **input** of the model, and what is its **output**? Give the exact shape of both.
2. What does "the model learned" mean, concretely, in terms of numbers inside the model?
3. Which metric would tell you the model works, and why is accuracy alone dangerous if there are twice as many dogs as cats?
4. Name three ways the model could be right for the wrong reason (hint: backgrounds, indoor or outdoor, lighting, a leash or a sofa).
5. Name two kinds of photo on which you expect the model to fail.

## Task 02 - Get the dataset

We use the **Oxford-IIIT Pet** dataset, which ships with torchvision. It contains about 7,300 photos of 37 breeds (12 cat breeds, 25 dog breeds). With `target_types="binary-category"` each label is simply `0` for cat and `1` for dog. The download is about 800 MB and takes a few minutes.

download

```
from torchvision import datasets

ROOT = "data"
trainval = datasets.OxfordIIITPet(root=ROOT, split="trainval",
                                  target_types="binary-category", download=True)
test = datasets.OxfordIIITPet(root=ROOT, split="test",
                              target_types="binary-category", download=True)

img, label = trainval[0]          # a PIL image and an int
print(len(trainval), len(test), img.size, label)
```

> **i Tip:** If the download is blocked from your network, use the Hugging Face dataset `microsoft/cats_vs_dogs` (25,000 photos) with the `datasets` library. A few of its files are corrupted: your loader must skip them without crashing, which is a good exercise.

**Delivery:** the cell above runs from a fresh Colab session, and the printed numbers are copied in your README.

## Task 03 - Explore like a scientist

Write a notebook section that produces, with your own code:

- ✓ the number of cats and dogs in `trainval` and in `test` (read the labels **without decoding every image**: look at the attributes of the dataset object with `dir(trainval)` and at the torchvision source),
- ✓ a bar chart of the class balance,

- ✓ the distribution of image widths and heights (are the images all the same size?),
- ✓ a grid of 16 random images with their label as title, using `matplotlib`,
- ✓ the **majority-class baseline**: the accuracy of a program that always answers the most frequent class.

Terminal

```
$> python3 explore.py
trainval: cats = ... dogs = ...
majority baseline accuracy: ...%
```

⚠ **Warning:** The classes are not balanced. Every result you obtain later must be compared with this baseline, otherwise a trivial program looks like a good model.

**Look at 30 images carefully.** Write 5 observations in your notebook: unusual angles, several animals in one photo, cropped faces, humans, backgrounds.

## Self-check (no notes!)

1. Why is accuracy misleading on unbalanced classes?
2. How many numbers describe one 128 x 128 colour image?
3. What could a model learn from the background instead of from the animal?

# Day 2 - Data pipeline

A model is only as good as what you feed it. Today you build a pipeline that is **correct** (no leakage between sets) and **reproducible** (same seed, same split).

## Task 04 - Train, validation, test

The dataset already gives `trainval` and `test`. Split `trainval` into **85% train** and **15% validation** with a fixed seed. Keep the index lists, you need them twice (see the warning).

src/data.py

```
import torch
from torch.utils.data import Subset

def split_indices(n, val_fraction=0.15, seed=42):
    g = torch.Generator().manual_seed(seed)
    idx = torch.randperm(n, generator=g).tolist()
    n_val = int(val_fraction * n)
    return idx[n_val:], idx[:n_val]          # train_idx, val_idx

train_idx, val_idx = split_indices(len(trainval))
assert set(train_idx).isdisjoint(val_idx)   # no image in both sets
print(len(train_idx), len(val_idx))
```

⚠ **Warning: Classic bug.** If you use `random_split` on one dataset object, training and validation share the same transforms, so your validation images get randomly flipped and colour-jittered too. Build **two** dataset objects with different transforms and apply the same index lists with `Subset`, as in Task 06.

**i Tip:** Is the split balanced? Compute the cat ratio in train and in validation. If they differ a lot, think about a stratified split and explain your choice in the README.

## Task 05 - Transforms and augmentation

A transform is a function applied to every image when it is loaded. Training and evaluation use **different** pipelines: random changes (augmentation) help the model generalise but would make validation and test results noisy.

src/data.py

```
from torchvision import transforms

IMG = 128
MEAN, STD = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]   # ImageNet statistics

train_tf = transforms.Compose([
    transforms.Resize((IMG, IMG)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])
eval_tf = transforms.Compose([
    transforms.Resize((IMG, IMG)),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])
```

- ✓ Explain in `DESIGN.md` what each line does (`ToTensor`: what changes in the shape and in the value range? `Normalize`: why subtract a mean?).
- ✓ Why do we resize to a square even though the photos have different ratios? What do you lose?
- ✓ Add one more augmentation of your choice (`RandomRotation`, `RandomResizedCrop`, ...) and justify it. Which augmentation would be **wrong** for this task?

## Task 06 - Datasets and DataLoaders

src/data.py

```
from torch.utils.data import DataLoader

def make_loaders(root, batch_size=32, num_workers=2):
    def ds(split, tf):
        return datasets.OxfordIIITPet(root=root, split=split,
                                      target_types="binary-category", transform=tf)
    train_ds = Subset(ds("trainval", train_tf), train_idx)
    val_ds = Subset(ds("trainval", eval_tf), val_idx)
    test_ds = ds("test", eval_tf)
    train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=...,
                              num_workers=num_workers)
    val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
                            num_workers=num_workers)
    test_loader = ...      # TODO: same as val
    return train_loader, val_loader, test_loader
```

- ✓ fill the two `TODO`: which loaders shuffle, and why?
- ✓ print the shape and dtype of one batch (expected: images `[32, 3, 128, 128]`, labels `[32]`),
- ✓ display a batch from the **train** loader twice and observe that the augmented images change; display one from the **val** loader twice and observe that they do not.
- ✓ note that you need to **undo** the normalisation before showing an image. Write that small function yourself.

⚠ **Warning:** Corrupted or unreadable images make a loader crash after 20 minutes of training. Iterate over each loader once, now, to prove that every image loads.

## Self-check (no notes!)

1. Why do we shuffle the training set but not the validation set?
2. What would happen to your validation accuracy if augmentation was applied to it?
3. What does `batch_size` change, in terms of memory and of the gradient estimate?

# Day 3 - Your first CNN

You write the architecture. It will not be the best network in the world, but it will be **yours**, with weights initialised randomly and trained by you.

## Task 07 - Shapes by hand

The size of a convolution output is `out = floor((in + 2*padding - kernel) / stride) + 1`. A max-pooling with `kernel=2` halves height and width.

- ✓ a 128 x 128 image goes through `Conv2d(3, 16, kernel_size=3, padding=1)`: what is the output shape? How many **parameters** does this layer have (weights and biases)?
- ✓ then `MaxPool2d(2)`: what shape now?
- ✓ repeat for 32 and 64 channels: what shape do you get before the classifier?
- ✓ how many parameters would a single `Linear` layer from the raw pixels to 64 neurons have? Compare with your first convolution and conclude.

**Delivery:** `DESIGN.md`, with the calculation shown. Check each value afterwards with code.

## Task 08 - Write SmallCNN

src/model.py

```
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self, n_classes=2):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            ...,        # TODO: block 2, 16 -> 32 channels
            ...,        # TODO: block 3, 32 -> 64 channels
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(..., 64),      # TODO: how many features after Flatten?
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, n_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

- ✓ why is there **no softmax** at the end of `forward`? (Look at what `nn.CrossEntropyLoss` expects in the documentation.)
- ✓ count the parameters with `sum(p.numel() for p in model.parameters())` and compare with your hand calculation.

## Task 09 - Sanity checks before training

Experienced engineers check the model **before** training. Run these three checks and write the result of each in your notebook.

sanity

```
model = SmallCNN().to(device)
x, y = next(iter(train_loader))
logits = model(x.to(device))
print(logits.shape)                        # expected: torch.Size([32, 2])
loss = nn.CrossEntropyLoss()(logits, y.to(device))
print(loss.item())                         # expected: close to 0.69
```

1. **Shape:** the logits have shape `[batch, 2]`.
2. **Initial loss:** with 2 balanced classes and random weights, the loss should be close to `ln(2) = 0.693`. Why? If you get `3.5` or `0.01`, something is wrong.
3. **Overfit one batch:** train on **one single batch** repeated 50 times. The loss must go to almost 0. If it cannot, you have a bug, and no amount of data will fix it.

**i Tip:** Wait for Task 11 to run check 3 if you prefer, but do not skip it. This habit saves days.

## Self-check (no notes!)

1. What does a convolution detect, and why does the same filter slide over the whole image?
2. Why must the number of inputs of the first `Linear` match the output of `Flatten`?
3. What does an initial loss of 0.69 tell you?

# Day 4 - Training

Here the model learns. You write the loop yourself: forward pass, loss, backward pass, update. Everything else in deep learning is a variation of these five lines.

## Task 10 - The training loop

Complete the skeleton. Each `TODO` is one line; write a comment explaining it in your own words.

src/train.py

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
EPOCHS = 15

for epoch in range(EPOCHS):
    model.train()
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        logits = ...        # TODO 1: forward pass
        loss = ...          # TODO 2: compare logits and labels
        ...                 # TODO 3: reset the gradients
        ...                 # TODO 4: backpropagation
        ...                 # TODO 5: update the weights
    val_loss, val_acc = evaluate(model, val_loader, loss_fn, device)
    print(f"epoch {epoch:2d}  val_loss {val_loss:.3f}  val_acc {val_acc:.3f}")
```

⚠ **Warning:** If you forget TODO 3, gradients accumulate from one batch to the next and training becomes erratic. Try it on purpose once, and note what you see.

## Task 11 - Evaluation function and metrics

Write `evaluate(model, loader, loss_fn, device)` returning the average loss and the accuracy. Inside it: `model.eval()` and `torch.no_grad()`. Explain in `DESIGN.md` why both are needed (Dropout and gradient memory are part of the answer).

src/train.py

```
@torch.no_grad()
def predict_all(model, loader, device):
    """Return labels and class probabilities for a whole loader."""
    model.eval()
    ys, probs = [], []
    for x, y in loader:
        p = torch.softmax(model(x.to(device)), dim=1)
        ys.append(y)
        probs.append(p.cpu())
    return torch.cat(ys), torch.cat(probs)
```

- ✓ record at each epoch the train loss, val loss, train accuracy and val accuracy,
- ✓ plot the two loss curves and the two accuracy curves in one figure,
- ✓ save the weights of the **best validation epoch** with `torch.save(model.state_dict(), ...)`.

## Task 12 - Experiments

Training is an experimental science: change **one thing at a time** and write down the result. Run at least 5 experiments and log them in `experiments.csv`.

| run | model | lr | batch | ep. | aug. | best val acc | notes |
|---|---|---|---|---|---|---|---|
| 1 | SmallCNN | 1e-3 | 32 | 15 | yes | ... | baseline |
| 2 | SmallCNN | 1e-2 | 32 | 15 | yes | ... | ... |
| 3 | SmallCNN | 1e-4 | 32 | 15 | yes | ... | ... |
| 4 | SmallCNN | 1e-3 | 32 | 15 | no | ... | effect of augmentation |
| 5 | SmallCNN | 1e-3 | 64 | 15 | yes | ... | effect of batch size |

> **i Tip:** Typical numbers for a small CNN on this task are in the range of 80% to 92% validation accuracy. Yours may differ. What matters is that you can explain **why** run 3 behaves differently from run 2.

**Delivery:** the curves of your best run in `README.md`, and one paragraph on what you observed about the learning rate.

## Self-check (no notes!)

1. What are the five steps of one training iteration?
2. What is the difference between `model.train()` and `model.eval()`?
3. Your training accuracy is 99% and your validation accuracy is 83%. What is happening?

# Day 5 - Honest evaluation

Today you open the test set, **once**. A single accuracy number is not enough: you want to know where the model is right, where it is wrong, and why.

## Task 13 - Test your best model

Load the best weights of your best run (chosen on **validation**), compute the predictions on the test set and report:

evaluation

```
from sklearn.metrics import confusion_matrix, classification_report

ys, probs = predict_all(model, test_loader, device)
preds = probs.argmax(dim=1)
print(confusion_matrix(ys, preds))
print(classification_report(ys, preds, target_names=["cat", "dog"], digits=3))
```

- ✓ accuracy, and the majority baseline from Day 1 next to it,
- ✓ the confusion matrix, drawn as a heat map with the numbers inside the cells,
- ✓ **precision** and **recall** for each class. Explain each in one sentence, with an example about cats.
- ✓ the difference between validation and test accuracy, and what it tells you.

⚠ **Warning:** If the test result is disappointing, you are **not allowed** to tune the model on the test set. You report it as it is. This is what separates an experiment from wishful thinking.

## Task 14 - Error analysis

Show the **16 most confident wrong predictions** (largest probability on the wrong class) in a grid, with the true label and the predicted probability in the title. Then classify each mistake in a table:

| Image | True | Predicted | Confidence | Likely cause |
|---|---|---|---|---|
| #412 | cat | dog | 0.97 | dark cat, no face visible |
| ... | ... | ... | ... | ... |

- ✓ group the causes (angle, lighting, breed that looks like the other class, bad label, several animals, ...),
- ✓ is there a **breed** with many errors? You can check it with `target_types="category"` on the same images,
- ✓ propose two concrete actions that would reduce the biggest group of errors.

## Task 15 - A stronger baseline (optional but recommended)

Train a **logistic regression** on the raw pixels of images resized to 32 x 32 (scikit-learn). Compare it with the majority baseline and with your CNN. What does the gap tell you about the value of convolutions?

## Self-check (no notes!)

1. Why do we not choose hyper-parameters with the test set?
2. Explain precision and recall for the class "cat", with an example.
3. What is the most frequent cause of errors in your model, and how would you verify it?

# Day 6 - Improve: regularisation and transfer learning

You now know what your model does badly. You improve it in two ways: you fight overfitting, then you start from a network that has already learned to see.

## Task 16 - Fight overfitting

Look at your curves. If validation loss rises while training loss falls, the model overfits. Test **at least two** of these remedies, one at a time, and log them in `experiments.csv`:

- ✓ stronger augmentation (`RandomResizedCrop`, `RandomRotation`),
- ✓ more `Dropout`, or `weight_decay` in the optimiser (for example `1e-4`),
- ✓ a smaller network, or fewer epochs with **early stopping** on validation loss,
- ✓ a learning rate scheduler (`torch.optim.lr_scheduler`).

**Delivery:** a table of the experiments and one paragraph: which remedy helped the most, and does it match what you expected?

## Task 17 - Transfer learning with ResNet18

A ResNet18 pretrained on ImageNet already knows edges, textures and shapes (ImageNet contains many cat and dog breeds). You keep its body and replace its last layer by a layer with 2 outputs. Use `IMG = 224` for this model.

src/model.py

```
from torchvision import models

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False                    # freeze the body
model.fc = nn.Linear(model.fc.in_features, 2)  # new head, trainable
model = model.to(device)

optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
# train with the SAME loop as Task 10, for about 5 epochs
```

1. Train only the new head (about 5 epochs). Record the validation accuracy.
2. **Fine-tune**: unfreeze the last block (`model.layer4`) with a small learning rate (`1e-4`), train 3 more epochs, and compare.
3. How many parameters are trainable in each setting? Print them.

⚠ **Warning:** Distinction to remember: your SmallCNN was **built from scratch**; the ResNet is **adapted from a pretrained model**. Its strong result comes from knowledge learnt on another very large dataset, and you must say so in your report.

## Task 18 - Final comparison on the test set

Choose your final model **using validation only**, then evaluate the final candidates on the test set one last time.

| Model | Trainable params | Val acc | Test acc | Train time | Comment |
|---|---|---|---|---|---|
| Majority baseline | 0 | … | … | 0 | always the frequent class |
| Logistic regression | … | … | … | … | optional |
| SmallCNN (yours) | … | … | … | … | from scratch |
| ResNet18, head only | … | … | … | … | pretrained |
| ResNet18, fine-tuned | … | … | … | … | pretrained |

## Self-check (no notes!)

1. Why does the pretrained model need less data to reach a good result?
2. What does `requires_grad = False` do?
3. Is it fair to compare your SmallCNN with a pretrained ResNet18? Argue both sides.

# Day 7 - Ship it

A model in a notebook helps nobody. Today you turn it into a tool, you confront it with the real world, and you document it honestly.

## Task 19 - predict.py

A command line program that loads your final weights and prints the answer for one or several images. It must run **without the training code** and without the dataset.

src/predict.py

```
import argparse
import torch
from PIL import Image

CLASSES = ["cat", "dog"]

def load_model(weights_path, device):
    ...          # TODO: rebuild the architecture, load_state_dict, eval()

def predict(model, image_path, device):
    ...          # TODO: open, convert("RGB"), eval_tf, softmax -> (label, prob)

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("images", nargs="+")
    ap.add_argument("--weights", default="models/best.pt")
    ap.add_argument("--threshold", type=float, default=0.8)
    args = ap.parse_args()
    ...          # TODO: print one line per image
```

Terminal

```
$> python3 src/predict.py photos/rex.jpg photos/blank.png
photos/rex.jpg: dog (probability ...)
photos/blank.png: unsure (probability ...)
$> python3 src/predict.py missing.jpg
Error: file 'missing.jpg' not found
```

- ✓ a missing or non-image file gives a clear message and exit code 1,
- ✓ if the top probability is below `--threshold`, print `unsure` instead of a class,
- ✓ `convert("RGB")` handles grayscale and PNG images with transparency.

## Task 20 - The real-world test

Build a folder `realworld/` with **photos you took or are allowed to use**:

- ✓ 20 photos of cats and dogs, taken in **your** conditions (phone camera, night, partly hidden animals, local street animals, puppies, several animals),
- ✓ 10 photos of **things that are neither**: a goat, a rabbit, a fox, a toy, a car, a chair, a person.
- ✓ run your model on all of them and report accuracy on the first group.

Then answer in `MODEL_CARD.md`: what does the model say about the 10 images that are neither? Does the threshold of Task 19 help? Try values between 0.5 and 0.95 and describe the trade-off.

⚠ **Warning:** A softmax always produces two probabilities that sum to 1, even for a car. A **confident wrong answer** is normal behaviour here, not a bug. A model only knows the classes it was trained on. Do not use photos of identifiable private persons without their consent.

## Task 21 - Demo app (bonus)

Give your model a face with Gradio (`pip install gradio`): upload a photo, see the two probabilities.

app.py

```
import gradio as gr

def classify(img):
    ...      # TODO: return {"cat": p_cat, "dog": p_dog}

demo = gr.Interface(fn=classify, inputs=gr.Image(type="pil"),
                    outputs=gr.Label(num_top_classes=2))
demo.launch()
```

**i Tip:** Hugging Face Spaces can host this demo for free, so you can share a link. This is the same mindset as the Legal-Benin application later in the piscine.

## Task 22 - Model card and README

Write `MODEL_CARD.md` with these sections, in your own words:

- ✓ **Purpose:** what it does, and what it must not be used for,
- ✓ **Data:** dataset, size, class balance, what it does not contain,
- ✓ **Model:** architecture, number of parameters, training recipe, seed,
- ✓ **Results:** test accuracy, confusion matrix, comparison table,
- ✓ **Limits:** failure cases from Task 14 and Task 20, with examples,
- ✓ **Next steps:** three realistic improvements.

`README.md`: how to install, how to reproduce your best run, how to use `predict.py`. A classmate must be able to run it from a fresh clone.

## Self-check (no notes!)

1. What is a threshold doing to the answers of your tool?
2. Why can the model be 99% confident about a photo of a goat?
3. What would you need to change to recognise a third class (rabbits)?

# Evaluation

| Criterion | Points |
|---|---|
| Data pipeline: correct split, no leakage, augmentation only on train | 15 |
| Own architecture and own training loop, sanity checks done | 20 |
| Honest evaluation: test used once, confusion matrix, error analysis | 20 |
| Improvement: regularisation experiments and transfer learning, clear comparison | 15 |
| Real-world test and handling of unknown images | 10 |
| Code quality, Git history, reproducibility | 10 |
| Model card and oral explanation without notes | 10 |
| **Total** | **100** |

## Oral defence (15 minutes)

- ✓ Draw your network on a board and give the shape after each layer.
- ✓ Explain one training iteration without looking at the code.
- ✓ Show one wrong prediction and explain what the model probably used.
- ✓ Explain why your test number is trustworthy, or why it might not be.
- ✓ Say what you would do with one more week.

## Final checklist

- ✓ the repository runs from a fresh clone, no secret and no weights in Git,
- ✓ `experiments.csv` contains every run, including the failures,
- ✓ the test set was evaluated exactly twice (Day 5 and Day 6), and you can prove it,
- ✓ `predict.py` works on an image and refuses a bad path,
- ✓ the model card states the limits before the successes.

## Glossary

| Term | Meaning |
|---|---|
| Tensor | A grid of numbers with any number of dimensions; an image batch is `[B, 3, H, W]`. |
| Logit | Raw score of the network before softmax. |
| Softmax | Turns scores into probabilities that sum to 1. |
| Cross-entropy | Loss that punishes a low probability on the true class. |
| Epoch | One full pass over the training set. |
| Batch | Group of samples used for one weight update. |
| Overfitting | Good on training data, worse on new data. |
| Augmentation | Random changes of training images to improve generalisation. |
| Transfer learning | Reusing a network pretrained on another task and adapting it. |
| Fine-tuning | Continuing training of some pretrained layers with a small learning rate. |

## Official documentation to read

- ✓ PyTorch tutorials, section "Learn the Basics": pytorch.org/tutorials
- ✓ torchvision datasets, transforms and models: pytorch.org/vision/stable
- ✓ scikit-learn metrics (confusion matrix, classification report): scikit-learn.org

**i Tip:** Next piscine: you will rebuild the ideas of this project at a lower level, neuron by neuron, then go further with Transformers.